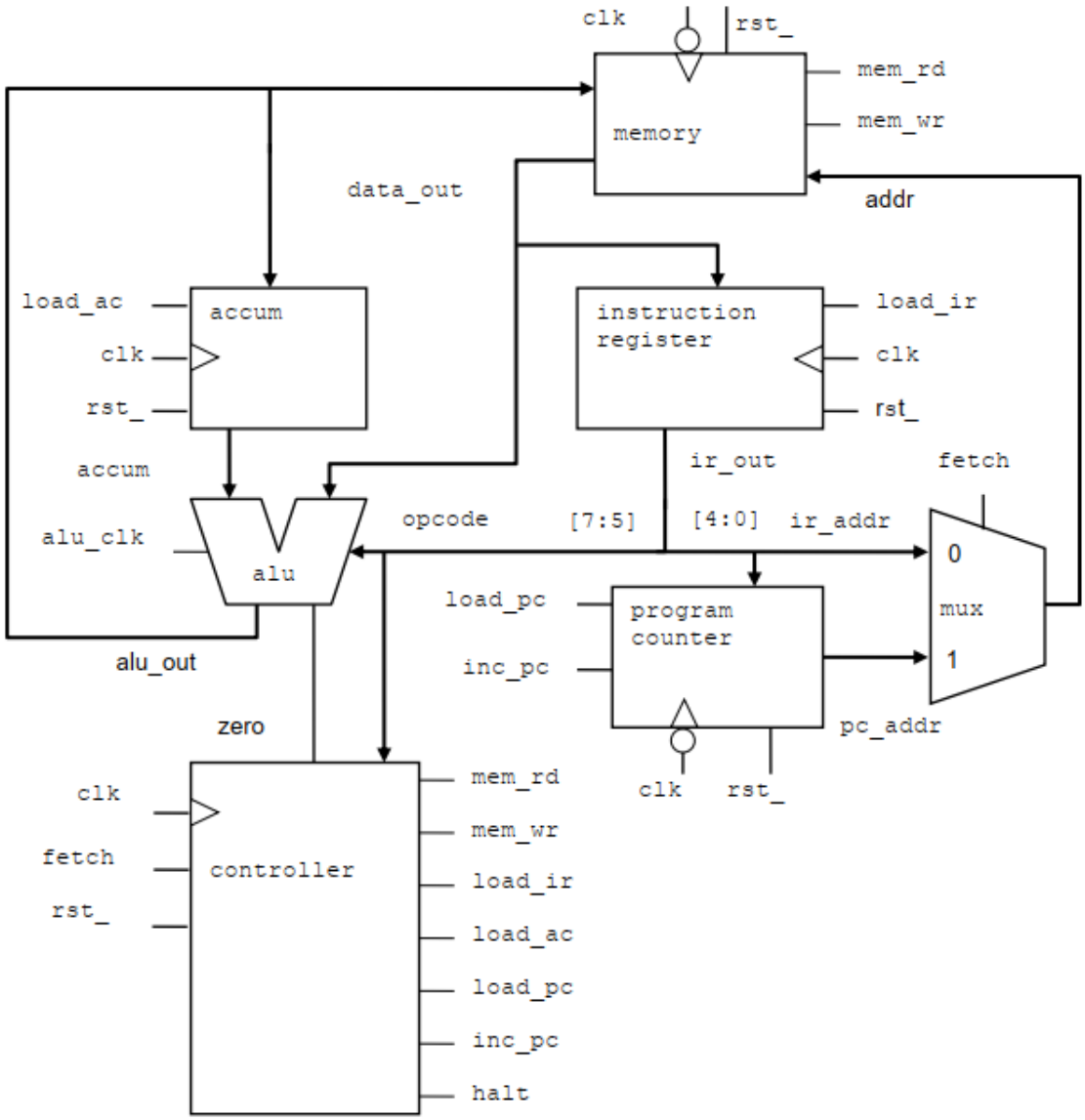
Assemble the blocks of the CPU using SystemVerilog connectivity enhancements, and test the design using the supplied testbench and diagnostic programs.

You can do this lab exercise any time after Lab 6.

Read the specification first and then follow the instructions in the lab section, *Assembling the CPU Model*.



Specification

The CPU architecture is as follows:

- The Program Counter (`counter`) provides the program address.
- The MUX (`scale_mux`) selects between the program address or the address field of the instruction.
- The Memory (`memory`) accepts data and provides instructions and data.
- The Instruction Register (`register`) accepts instructions from memory.
- The Accumulator Register (`register`) accepts data from the ALU.
- The ALU (`alu`) accepts memory and accumulator data and the opcode field of the instruction and provides new data to the accumulator and memory.

If all components of the CPU are working properly, it will:

- Fetch an instruction from memory.
- Decode the instruction.
- Fetch a data operand from memory if required by the instruction.
- Execute the instruction, processing mathematical operations, if required.
- Store results back into either the memory or the accumulator.

This process is repeated for every instruction in a program until an `HLT` instruction is found.

Assembling the CPU Model

Working in the `lab08-cpu` directory, perform the following.

1. Review the `cpu.sv` file, containing the `cpu` module declaration, which instantiates and connects all the components of the CPU.

2. Copy all the components from the `sv_src` directory into `lab08-cpu`. If you did not complete all the previous labs, you can find components in the `sv_src/files` directory.

3. Make sure that your package containing the enumerated type declaration for `opcode_t` is also copied into `lab08-cpu`, and make sure the package is imported into the `cpu` module.

Reviewing the CPU Testbench

4. Review the supplied testbench in the file `cpu_test.sv`. The testbench verifies your CPU design using three diagnostic programs as follows:
   - The testbench displays a message requesting the number of the diagnostic program and waits for user input.
   - When the user enters a test number and continues the simulation, the testbench loads the specified test microcode, resets the CPU, and outputs debug messages while it waits for the CPU to indicate the end of test by asserting the `halt` signal.
   - When `halt` is received, the testbench verifies that the Program Counter address is correct for the given test. If the address is incorrect, the test fails.
   - The testbench then re-displays the request for test message, and again waits for user input.

CPU Diagnostic Programs

- `CPUtest1.dat` – **Basic Diagnostic Test**: This program tests several of the VeriRISC instructions. If all the instructions execute correctly, the CPU will encounter an `HLT` instruction at Program Counter address (`cpu.pc_addr`) `0x17`. If the CPU halts at some other address, examine the `CPUtest1.dat` program file to determine which instruction failed. This file is well commented, so you can see what is supposed to happen.

- `CPUtest2.dat` – **Advanced Diagnostic Test**: This program loads and runs the advanced diagnostic program, which tests additional VeriRISC instructions. If all the instructions execute correctly, the CPU will encounter an `HLT` instruction at Program Counter address (`cpu.pc_addr`) `0x10`. If the CPU halts at some other address, examine the `CPUtest2.dat` file to determine which instruction failed.

- `CPUtest3.dat` – **The Fibonacci Calculator**: This test loads and runs a program that calculates the Fibonacci number sequence from 0 to 144 and stores the results in memory. If all the instructions execute correctly, the CPU will encounter an `HLT` instruction at Program Counter address (`cpu.pc_addr`) `0x0C`. If the CPU halts at some other address, examine the `CPUtest3.dat` file to determine which instruction failed.

Testing the CPU

5. Compile and simulate the CPU design and testbench.

   You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option:

   ```
   xrun -f filelist.txt -access rwc
   ```

   You will also need the **-access rwc** simulator option to set the test number.

6. Enter the following commands when you get the prompt:

   ```
   ncsim>  deposit test_number 1; run
   ncsim>  deposit test_number 2; run
   ncsim>  deposit test_number 3; run
   ```

7. You see the microcode instructions displayed as the VeriRISC CPU executes them, as in this example.

   ```
   CPUtest1 - BASIC CPU DIAGNOSTIC PROGRAM
      THIS TEST SHOULD HALT WITH THE PC AT 17 hex
       TIME       PC    INSTR    OP   ADR   DATA
      -------     --    -----    --   ---   ----
        115ns     00    JMP      7    00     fe
        . . .
        . . .
       1475ns     17    HLT      0    17     00
       TIME       PC    INSTR    OP   ADR   DATA
      -------     --    -----    --   ---   ----
      CPU TEST 1 PASSED
   ```

   Debug your subprograms as required, until you are happy that the design is verified.

8. Rerun the simulation to verify that it runs successfully.